

Neural Network Attacks

Adversarial Examples

Machine Learning and Security

Fabio De Gaspari
Dorjan Hitaj

Attacks on DNN: Threat Model

Adversarial attacks are DNN attacks aimed at making a ***target model*** misbehave in one of several possible ways.

Typically consider two distinct threat models

- Whitebox: Attacker has full knowledge of the model
- Blackbox: Attacker has no knowledge of the model

Attacks on DNN: Whitebox Threat Model

In a whitebox threat model:

- The attacker knows the training data distribution for the target model and input features used
  - e.g., images of wild animals
- The attacker knows the exact architecture of the target model
  - number of layers, number of neurons, activation functions, layer type, ...
- The attacker knows the exact parameters of the trained model
  - model weights, biases

Adversarial Examples

Evasion: perturb an input at test time so that the model misclassifies it

- **Goal**: misclassification, i.e., violate the integrity of the model's output
  - *untargeted*: the model assigns *any* wrong class
  - *targeted*: the model assigns a *specific* class t, chosen by the attacker
- **Capability**: can only modify the input, not the model or its training data
  - the perturbation is constrained so that it is non-obvious to observers, especially humans
- **Setting**: first whitebox, with gradient-based attacks; then blackbox

Adversarial Examples - Untargeted Attacks

Untargeted Adversarial Examples

Many different existing techniques to generate the perturbation

- typically, they solve one of two optimization problems. Minimum norm:

$$\delta^* = arg\ min_\delta \|\delta\| \quad s.t. \quad f(x+\delta) \neq f(x)$$

or, fixed budget: the strongest perturbation within distance ε:

$$\delta^* = arg\ max_{\|\delta\| \leq \epsilon}\ J(x+\delta, \theta)$$

where ||·|| is an Lp norm. C&W: minimum norm; FGSM, PGD: fixed budget

Untargeted Adversarial Examples

$$\delta^* = arg\ min_\delta \|\delta\| \quad s.t. \quad f(x+\delta) \neq f(x)$$

Where:

- *f* is a trained DNN
- x is the unperturbed sample
- δ is the adversarial perturbation
- $\tilde{x} = x + \delta^*$ is the adversarial example

Untargeted Adversarial Examples

$$\delta^* = arg\ min_\delta \|\delta\| \quad s.t. \quad f(x+\delta) \neq f(x)$$

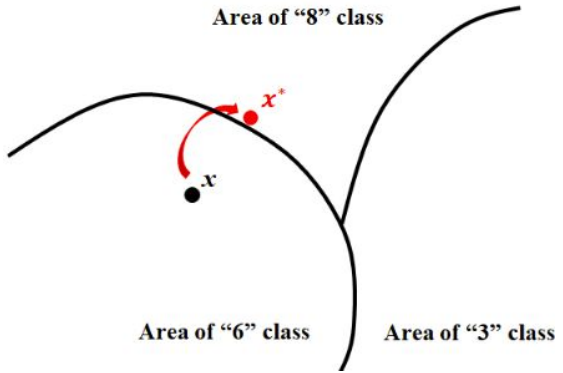
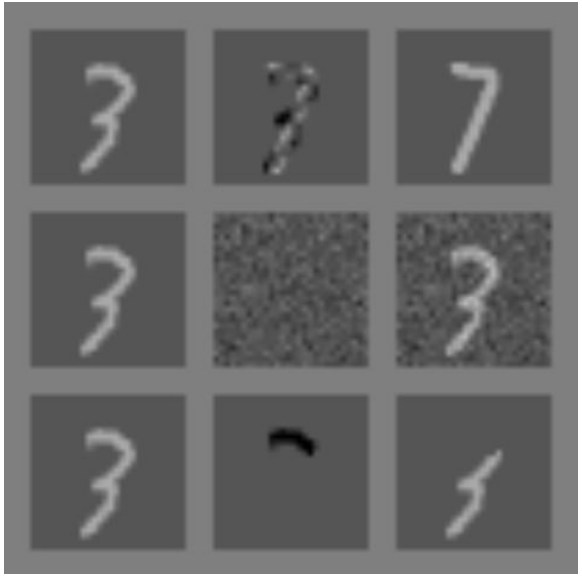


image from Kwon, Hyun, Kyoungmin Ko, and Sunghwan Kim. "Optimized adversarial example with classification score pattern vulnerability removed." IEEE Access 10 (2021)

Untargeted Adversarial Examples

How (not) to create adversarial examples



All these perturbations have the same L2 norm (euclidean distance)

Untargeted Adversarial Examples

How to create adversarial examples:

- changes need to be imperceptible and should not change the class of the sample for human (like the first row of the previous image)
- typically we do this by limiting the L-infinity (L-inf) norm, which is the largest magnitude among all elements of a vector:

$$X = [-5, 2, 3];\ \|X\|_\infty = 5$$

Fast Gradient Sign Method Attack

The Fast Gradient Sign Method (FGSM) is based on the *first order approximation*:

$$J(x + \delta, \theta) \approx J(x, \theta) + \delta^T \nabla_x J(x, \theta)$$

- Where x is the clean sample and x + δ is the perturbed sample

i.e., around x, the change in the loss is approximately the dot product between the perturbation δ and the gradient of the loss wrt the input

- so the attacker increases the loss the most by aligning the perturbation with the gradient
- J(x, θ): loss of the model with parameters θ on input x (its label y is left implicit)

Fast Gradient Sign Method Attack

Using this approximation, FGSM maximizes:

$$arg\ max_{\delta}\ J(x,\theta) + \delta^T \nabla_x J(x,\theta)$$

subject to: $\|\delta\|_\infty \leq \epsilon$

In the above maximization, we can only influence the second term through δ

- for this linear objective, the maximizer has a closed form: each term $\delta_i g_i$ (g = gradient wrt x) is maximized independently under $|\delta_i| \leq \varepsilon$, by $\delta_i = \varepsilon \cdot sign(g_i)$

$$arg\ max_{\|\delta\|_\infty \leq \epsilon}\ \delta^T \nabla_x J(x,\theta) = \epsilon\ sign(\nabla_x J(x,\theta))$$

Fast Gradient Sign Method Attack

Summarizing, FGSM uses gradient wrt x to compute the adversarial example as:

$$\tilde{x} = x + \delta = x + \epsilon \; sign(\nabla_x J(x, \theta))$$

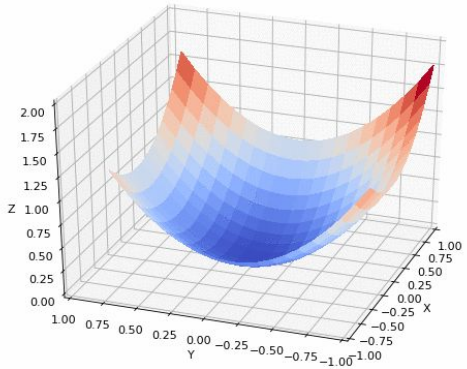
- FGSM is a *single step* method, since we only need to compute the gradient once to get the adv. example
- It is *untargeted*, since it moves the sample towards the direction of steepest increase of the loss, regardless of the class
- It is very fast and efficient: easy to implement

Fast Gradient Sign Method: Intuition

Remember Gradient Descent and Backpropagation?

FGSM is effectively applying a single-step of ***gradient ascent***, to increase the loss, with the gradients computed wrt the *input features*, rather than the weights

- the attacker has no control over the model itself, can only change the input
- this is the core idea behind all gradient-based methods for adversarial example generation



Projected Gradient Descent Attack

A problem with FGSM is that the single-step approach in the direction of the gradient is optimal only if the decision boundary is linear near the sample

- it is optimal vs linear classifiers
- even over relatively small regions, NNs are not linear

Projected Gradient Descent (PGD), closely related to Iterative FGSM (IFGSM), addresses this by using a multi-step optimization approach to maximize the loss

Projected Gradient Descent Attack

$$\delta^{(k+1)} = arg\ max_{\delta}\ J(x + \delta^{(k)}, \theta) + (\delta - \delta^{(k)})^T \nabla_x J(x + \delta^{(k)}, \theta)$$

subject to

$$\underbrace{\|\delta - \delta^{(k)}\|_\infty \leq \alpha}_{\text{step size } \alpha}, \quad \underbrace{\|\delta\|_\infty \leq \epsilon}_{\text{budget}}, \quad \underbrace{x + \delta \in [0, 1]^n}_{\text{box constraint}}$$

Similarly to FGSM, the solution has a closed form:

$$\delta^{(k+1)} = \mathcal{P}_\epsilon \left\{ \delta^{(k)} + \alpha\ sign\left(\nabla_x J(x + \delta^{(k)}, \theta) \right) \right\}$$

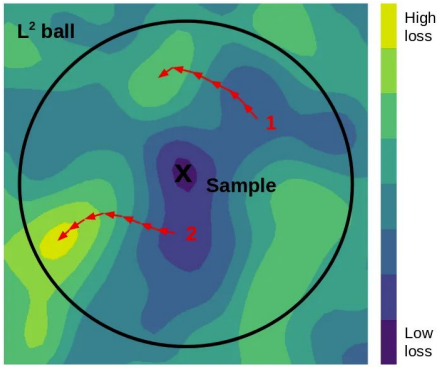
where P_ε is a *projection operator*: clips δ to [-ε, ε] and x + δ to [0, 1] (box constraint)

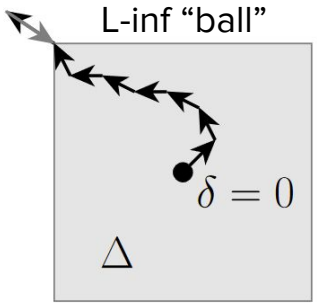
Projected Gradient Descent Attack

$$\delta^{(k+1)} = \mathcal{P}_\epsilon \left\{ \delta^{(k)} + \alpha \; sign \left(\nabla_x J(x + \delta^{(k)}, \theta) \right) \right\}$$

δ^0 can be initialized as:

- 0 (start from the clean sample x). This is typically called IFGSM
- random noise in [-ε, ε]: x + δ^0 starts at a random point in the L-inf ball around x. This is PGD





Projected Gradient Descent Attack

$$\delta^{(k+1)} = \mathcal{P}_\epsilon \left\{ \delta^{(k)} + \alpha \; sign \left(\nabla_x J(x + \delta^{(k)}, \theta) \right) \right\}$$

When δ^0 is initialized randomly, we can also add *random restarts* to PGD

- As with NN training with random weight initialization, restarting PGD can help avoid local optima and converge to a better attack

Adversarial Examples - Targeted Attacks

Targeted Adversarial Examples

Targeted adv. examples add the additional constraint of forcing the model to misclassify a sample to a specific target class

- they solve the targeted versions of the same two problems. Minimum norm:

$$\delta^* = arg\ min_\delta \|\delta\| \quad s.t. \quad f(x+\delta) = t$$

or, fixed budget (minimize the loss wrt the target class t):

$$\delta^* = arg\ min_{\|\delta\| \leq \epsilon} J(x+\delta, t, \theta)$$

where *t* is the target class that we want the model to misclassify the input as

Carlini-Wagner Attack

$$\delta^* = arg\ min_\delta\ \|\delta\| \quad s.t. \quad f(x+\delta) = t$$

The above equation is a very difficult, non-linear optimization problem since *f* (the NN) can be arbitrarily complex

- Carlini and Wagner solved this problem by introducing an equivalent constraint that is easier to optimize
- they also introduce a regularization term in the optimization, rather than an explicit constraint on the perturbation

$$\delta^* = arg\ min_\delta\ \|\delta\| \quad s.t. \quad C(x+\delta) \leq 0$$

which can be rewritten as

$$\delta^* = arg\ min_\delta\ \|\delta\| + \lambda\, C(x+\delta)$$

Carlini-Wagner Attack

$$\delta^* = arg\ min_\delta\ \|\delta\| + \lambda\, C(x+\delta)$$

We want to find δ that minimizes:

- the norm of δ: this is the regularization factor that minimizes the distance between the original sample x and the perturbed sample x + δ
- C(x + δ): equivalent constraint such that f(x + δ) = t iff C(x + δ) ≤ 0
  - then, minimizing the above equation naturally gives a perturbation that *f* classifies as *t*
- C() provides an approximated, optimizable reformulation of the original problem

Carlini-Wagner Attack

$$\delta^* = arg\ min_\delta\ \|\delta\| + \lambda\, C(x+\delta)$$

the authors suggest to use the following as *C*

$$C(x) = max(max\,\{Z(x)_i : i \neq t\} - Z(x)_t, 0)$$

where *t* is the target class and *Z* are the logits of the network: the output of the last layer of the network before the final activation function. Putting it all together:

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\,(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0)$$

The optimization can be now done using gradient descent

Carlini-Wagner Attack: Intuition

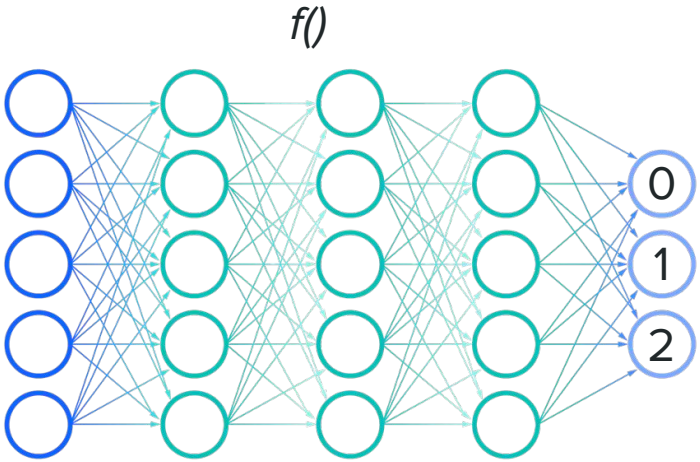
$$arg\ min_{\delta}\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0\right)$$

The function *C* computes:

1. in the inner *max* function, the maximum scalar value from the output of the network, among all classes *but* the target class (i ≠ t)
   - i.e., how much the model thinks x + δ belongs to the most likely class other than t
   - the difference between the inner max and $Z(x + \delta)_t$ is how much more the model prefers that class i over the target class t
2. the outer max function, between 0 and the difference between the output for the target class *t*, and the output from point 1 above
   - the outer max stops the optimization over the logits once argmax(Z) = t, i.e., once the model classifies the perturbed sample as t: from then on, only the norm of δ is minimized

Carlini-Wagner Attack: Intuition

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0\right)$$



Assume *t=2*; true class 0; *Z(x+δ) = [0.6, 0.3, 0.1]*

Then:

- max{Z(x+δ)$_i$: i ≠ t} = 0.6; Z(x+δ)$_t$ = 0.1
- difference = 0.5; C = max(0.5, 0) = 0.5

After some optimization, *Z(x+δ) = [0.2, 0.3, 0.5]*

Then:

- max{Z(x+δ)$_i$: i ≠ t} = 0.3; Z(x+δ)$_t$ = 0.5
- difference = -0.2; C = max(-0.2, 0) = 0
- we are now optimizing only ||δ||

Carlini-Wagner Attack: Intuition

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ -\kappa\right)$$

In the outer max, 0 can be replaced by -κ ($\kappa \geq 0$): the optimization continues until Z_t exceeds every other logit by at least κ

- meaning x is classified as class *t* with higher confidence; high-confidence adv. examples also transfer better to other models (useful in the blackbox setting, see later)

Carlini-Wagner Attack: Improving Box Constraint

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0\right)$$

Since we're dealing with images, x + δ must be bound in [0, 1] (normalized data)

- this is called *Box Constraint*. In PGD, we implement this through clipping in [0,1]
  - can work poorly with complicated update steps and limit the exploration space
- CW uses a change of variables with *tanh()* to natively include the constraint

$$x_i + \delta_i = 1/2\left(tanh(u_i) + 1\right) \ \Rightarrow\ \delta_i = 1/2\left(tanh(u_i) + 1\right) - x_i$$

  - *tanh()* is in (-1, 1), so with the new variable u: 0 ≤ x+δ = ½ (tanh(u) + 1) ≤ 1

The final optimization, over u and with the L2 norm as distance, is:

$$arg\ min_u\ \|1/2\left(tanh(u) + 1\right) - x\|_2^2 + \lambda\, C\left(1/2\left(tanh(u) + 1\right)\right)$$

Carlini-Wagner Attack

Question: can we make this attack untargeted?

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0\right)$$

Carlini-Wagner Attack

Question: can we make this attack untargeted?

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(max\{Z(x+\delta)_i : i \neq t\} - Z(x+\delta)_t,\ 0\right)$$

Yes: use the true class *y* instead of *t* and flip the difference, so that *any* class other than *y* works

$$arg\ min_\delta\ \|\delta\| + \lambda\ max\left(Z(x+\delta)_y - max\{Z(x+\delta)_i : i \neq y\},\ 0\right)$$

Blackbox Adversarial Examples

Attacks on DNN: Blackbox Threat Model

In a blackbox threat model:

- The attacker can query the target model in an “oracle” fashion:
  - send a query and only observe its output
- The attacker doesn’t know the architecture of the target model
- The attacker doesn’t know anything about the parameters of the target model

Attacks on DNN: Blackbox Threat Model

Typically two approaches to craft adv. examples in blackbox setting:

1. use *model extraction* to train a local, *surrogate* model. Craft adv. examples against the surrogate model, and rely on the *transferability* property hoping it will work on the target model as well
2. use *query-based* attacks: exploit the oracle outputs directly (e.g., to estimate gradients)

We will see 1 in detail; for 2, we will give a short overview of *score-based* and *decision-based* attacks at the end of this section

Surrogate Model

To create a surrogate model, we need to:

- choose an appropriate architecture for the task
- generate a synthetic dataset to train the surrogate model
- limit the number of queries to the oracle (the target model)

After we obtain the surrogate model, we can apply whitebox, gradient-based approaches to generate adv. examples (FGSM, PGD, ...)

- we then rely on the *transferability*[1] of adv. examples: an attack crafted on a given model (e.g., our surrogate) often also works on other models (target)

1 - Demontis, Ambra, et al. "Why do adversarial attacks transfer? explaining transferability of evasion and poisoning attacks." 28th USENIX security symposium. 2019.

Surrogate Model

1 - Choose an appropriate architecture for the task

- typically easy step, as the attacker has at least some basic knowledge of the target model (type of input, type of output)
- the attacker uses this knowledge to select an appropriate architecture
  - e.g., convolutional neural network for image classification
- Papernot et al. report that the type, number and size of the layers in the surrogate have limited impact on the attack success rate
- can also train multiple surrogate models and select the one yielding best results

Surrogate Model

2 - generate a synthetic dataset to train the surrogate model

- in principle, we could perform an infinite number of queries to the oracle, covering entirely the input domain.
  - this would allow us to train an exact copy of the target model
  - impractical and expensive
- efficiently sample the input distribution according to some appropriate heuristics, and query the oracle only on these sampled points
  - several approaches can be used here
  - we will see one approach proposed in: Papernot, Nicolas, et al. "Practical black-box attacks against machine learning." Proceedings of the 2017 ACM on Asia conference on computer and communications security. 2017.

Surrogate Model: Sample Input Distribution

Idea: identify the directions where the model's output varies around an initial set of points

1. given a small set of initial inputs, query the oracle on each input to create an initial dataset D_0
2. train the surrogate model over the current dataset D_i
3. augment the input set through *Jacobian-based Dataset Augmentation*, and query the oracle to label the new points

Repeat steps 2-3 until satisfactory results are obtained

Jacobian-based Dataset Augmentation

The main idea is to use the gradients of the surrogate model F to generate additional points in a smart way

- we take the gradient of the surrogate's score for one class (not of a loss, as in FGSM):

$$\nabla_x F_y(x), \quad y = O(x)$$

- this is one row of the Jacobian J_F of F, the matrix of all its partial derivatives
  - a large j-th component means that changing the j-th feature of x has a large impact on the score of class y

Jacobian-based Dataset Augmentation

Directions in the input space causing large changes in output probabilities likely lead towards the model's *decision boundary*

- boundary that separates the prediction between different classes
  - implying that the output probability also changes considerably
- we are essentially using the Jacobian to generate new samples that progressively push and update the decision boundary, refining the model

Jacobian-based Dataset Augmentation

Goal: match the decision boundary of the unknown model

- we evaluate the row of the Jacobian corresponding to the class assigned to a given input
  - i.e., find the direction in input space where the surrogate's output for that class changes the most: the most informative direction to explore where the oracle's labels might change
  - the class is obtained by querying the oracle
- then, we perturb the original point accordingly:

$$D_{i+1} = D_i \cup \{x + \lambda \; sign(J_F(x)[O(x)]) : x \in D_i\}$$

Jacobian-based Dataset Augmentation

$$D_{i+1} = D_i \cup \{x + \lambda \; sign(J_F(x)[O(x)]) : x \in D_i\}$$

Where:

- J_F is the Jacobian of the surrogate model F on point x
- O is the oracle (the target model that we can query), and y = O(x) is the classification of point x by the oracle
- λ is the step size (Papernot et al. periodically flip its sign)

The idea is that we want to move the current input x in the direction of maximum change wrt the output variable y

- effectively, this procedure iteratively refines the decision boundary of the surrogate model based on the output of the oracle

Query-based Attacks

The oracle returns output scores (e.g., class probabilities), but not gradients

- Estimate the gradient with finite differences, one coordinate at a time (ZOO[1]):

$$\hat{g}_i = \frac{\partial J(x)}{\partial x_i} \approx \frac{J(x + h\, e_i) - J(x - h\, e_i)}{2h}$$

  - e_i: one-hot vector selecting input feature i, h: small step; then plug the estimate into C&W/PGD
  - 2 queries per coordinate: expensive for high-dimensional inputs such as images
- Gradient-free alternatives use random search (e.g., Square Attack[2])
- *Decision-based* attacks (label only) also exist, but need far more queries[3]

1 - Chen, Pin-Yu, et al. "ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models." AISec 2017. 2 - Andriushchenko, Maksym, et al. "Square Attack: a query-efficient black-box adversarial attack via random search." ECCV 2020. 3 - Chen, Jianbo, et al. "HopSkipJumpAttack: a query-efficient decision-based attack." IEEE S&P 2020.

Adversarial Examples - Why They Exist

Why Do Adversarial Examples Exist?

Is it a problem of overfitting?

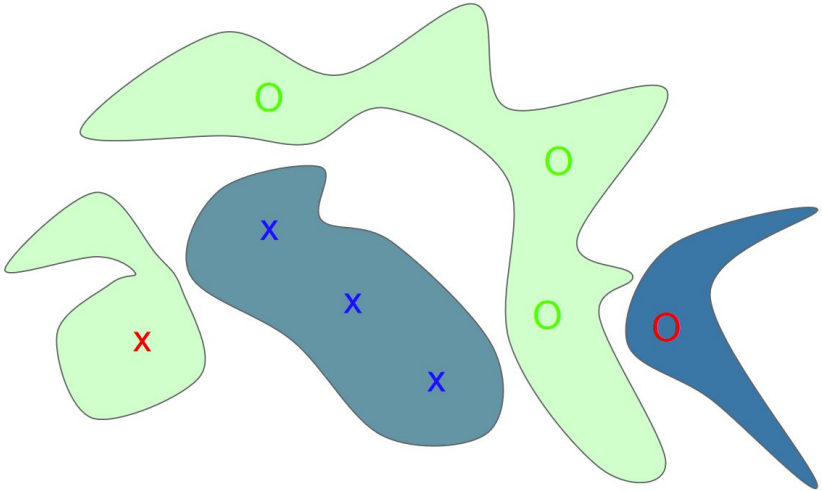


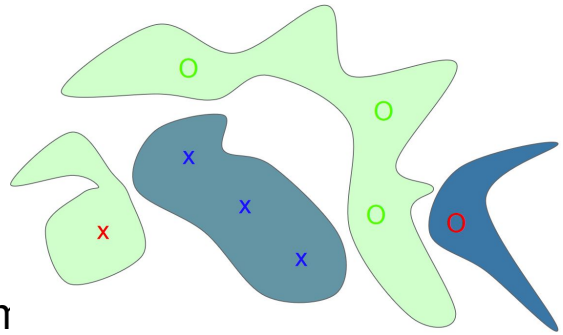
image from I. Goodfellow

model trained on green circles and blue samples

shaded areas are the boundary learned by the (overfitting) model

the red X and O are misclassified, are they adversarial examples?

Why Do Adversarial Examples Exist?



Is it a problem of overfitting?

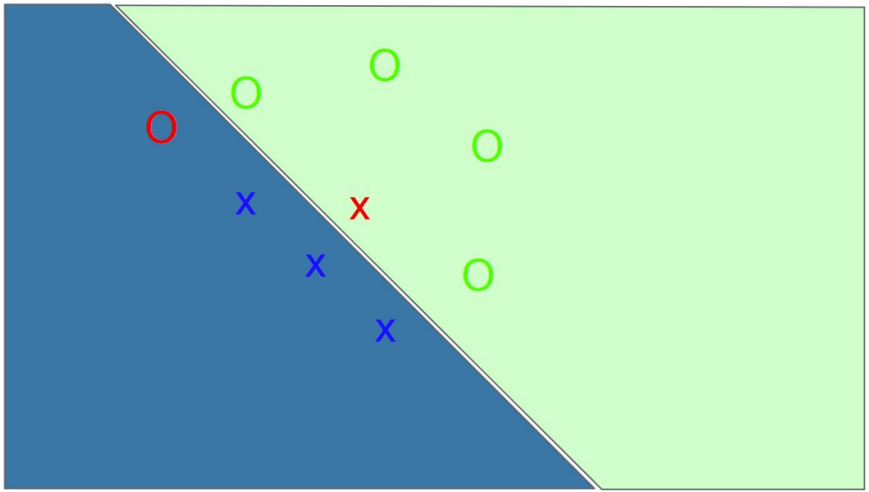
- then adv. examples would result from “bad luck”
  - the model happened to overfit that class in that region of space
- refitting a different model should make different n

This is not the case

- many different models are vulnerable to the same adv. example (transferability)
  - typically assign the same class to the example
- low capacity models would not suffer from the attack
  - but they actually have the same problem

Why Do Adversarial Examples Exist?

Excessive linearity?

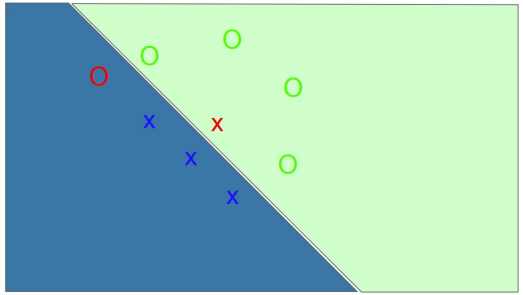


- DNNs are clearly not linear
  - but they have many linear components and are *piecewise* linear

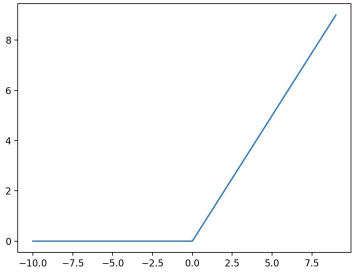
Why Do Adversarial Examples Exist?

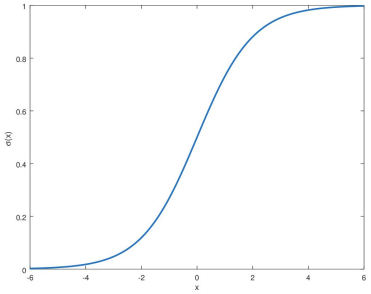
Linear models assign high confidence to areas where there are no datapoints at all

- see the top right/bottom left corners of the picture



DNNs use many linear components in the function composition



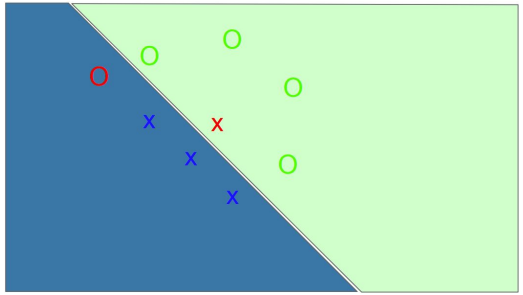


center part is close enough to linear

Why Do Adversarial Examples Exist?

Excessive linearity?

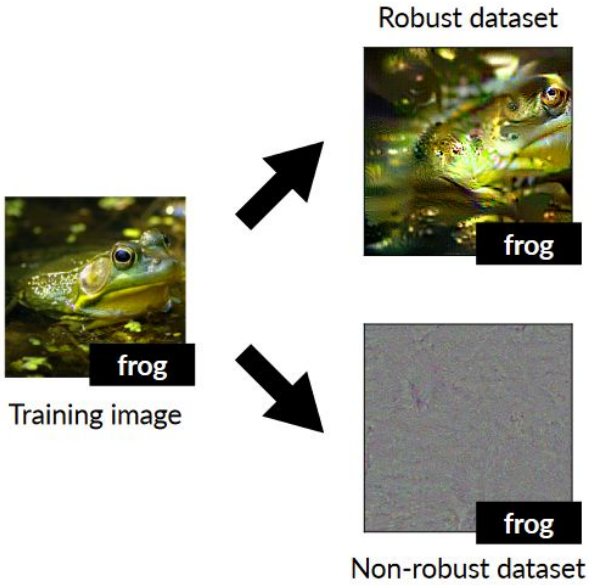
- no greater robustness observed in less linear models
- not fully compatible with other observations[1]



1 - Madry, Aleksander, et al. "Towards deep learning models resistant to adversarial attacks." ICLR 2018

Why Do Adversarial Examples Exist?

Brittle features?



Ilyas, Andrew, et al. "Adversarial examples are not bugs, they are features." *Advances in neural information processing systems* (2019)

Why Do Adversarial Examples Exist?

Brittle features?

- datasets are non-comprehensive
- some features might appear highly predictive, but are actually just defects of the dataset that happen to work for that specific setting
  - models are trained only to maximize the distributional accuracy. Any available signal in the dataset that helps correctly classify the training set, is used to reach this goal
  - from the point of view of the model, any highly predictive feature (even if meaningless for humans) can be used to achieve the minimization objective

Consistent with other previous explanations, such as overfitting

Why Do Adversarial Examples Exist?

No clear consensus: several hypotheses, each with evidence for and against

Hypothesis	Evidence for	Evidence against
Overfitting	intuitive: models err where they saw no data	transferability; low-capacity models are vulnerable too
Excessive linearity	FGSM, a linear attack, works well; DNNs are piecewise linear	less linear models are not more robust
Non-robust features	models trained only on non-robust features generalize; explains transferability	still debated; may not explain all adv. examples

Common thread: models use any signal that lowers the loss, including signal humans cannot perceive, so robustness has to be built into training ➜ defenses

Adversarial Examples - Defenses

Adversarial Training

Currently, the state of the art defense against adv. examples is *adversarial training*

Idea: make model more robust by incorporating adv. examples in training set

- this changes the min optimization problem of training

$$arg\ min_\theta\ J(x, \theta)$$

- to a more complex min-max optimization problem:

$$arg\ min_\theta\ \ max_{\|\delta\|_\infty \leq \epsilon}\ J(x + \delta, \theta)$$

- where δ is computed separately for each training sample

Adversarial Training

$$arg\ min_\theta\ \ max_{\|\delta\|_\infty \leq \epsilon}\ J(x+\delta, \theta)$$

This problem is non-convex in general (a saddle-point problem), however in practice it is shown to converge under normal training with SGD

- Danskin's Theorem:

$$\nabla_\theta\ max_{\|\delta\|_\infty \leq \epsilon}\ J(x+\delta, \theta) = \nabla_\theta J(x+\delta^*, \theta)$$

where $\delta^* = arg\ max_{\|\delta\|_\infty \leq \epsilon}\ J(x+\delta, \theta)$

Adversarial Training

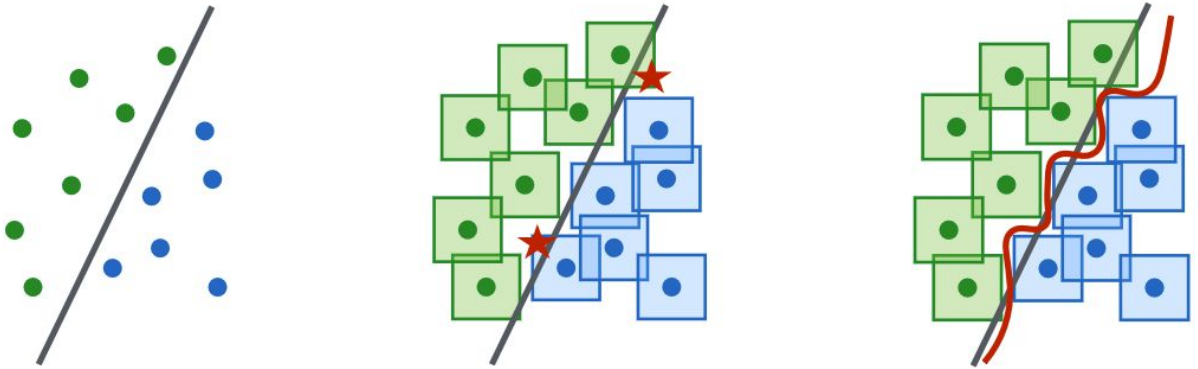
$$\nabla_\theta \, max_{\|\delta\|_\infty \leq \epsilon} \, J(x + \delta, \theta) = \nabla_\theta J(x + \delta^*, \theta)$$

$$\delta^* = arg \, max_{\|\delta\|_\infty \leq \epsilon} \, J(x + \delta, \theta)$$

In practice, this is implemented by using an adv. example generation algorithm (e.g., PGD) within the training loop to find δ^*

- each batch of samples is perturbed and adv. examples are generated
- we compute the loss on the adv. examples
- backpropagate the gradient of the loss to update the model and optimize θ
  - typically also mix in some clean samples together with adversarial samples

Adversarial Training: Intuition



The dots are datapoints, the squares represent the allowed adv. perturbation. The gray line is the standard decision boundary, the red is the robust decision boundary. Read cited paper for more details on adv. training

from Madry, Aleksander, et al. "Towards deep learning models resistant to adversarial attacks." ICLR 2018

Adversarial Training

Adversarial Training can typically help in improving the resilience of a model to adv. attacks, but has drawbacks:

- only improves resilience, it is still possible to find effective adv. examples
- considerably increases training time, as it solves a multi-step optimization problem for each training step (FGSM/PGD/CW/other attack)
- reduces accuracy on clean samples (robustness vs. accuracy trade-off)
- robustness only holds for the threat model used in training (norm, ε): e.g., patches or unconstrained attacks (Part 2) can still succeed

Evaluating Defenses: Gradient Masking

Many defenses looked robust only because they broke the attack

- *Gradient masking*: defenses that make gradients useless (non-differentiable steps, randomness, vanishing gradients)
  - 7 of 9 ICLR 2018 defenses relied on it; adaptive attacks broke 6 fully and 1 partially[1]
  - red flags: blackbox/transfer attacks beat whitebox ones, more PGD iterations do not help
- Good practice: adaptive attacks designed for the specific defense, standard benchmarks (AutoAttack[2])
- Certified defenses (e.g., randomized smoothing[3]) prove robustness within a radius, at a cost in accuracy

1 - Athalye, Carlini, Wagner. "Obfuscated gradients give a false sense of security." ICML 2018. 2 - Croce, Hein. "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks." ICML 2020. 3 - Cohen, Rosenfeld, Kolter. "Certified adversarial robustness via randomized smoothing." ICML 2019.

Recap: Assumptions So Far

So far, we assumed that:

- the perturbation is small: bounded in an Lp norm ($||\delta|| \leq \varepsilon$), so it is imperceptible
- each perturbation is crafted for a single input
- input features are continuous and can be freely modified (e.g., pixel values)
- the attacker feeds the perturbed input directly to the model (digital domain)

Next (Part 2): what happens when we relax these assumptions?

- unrecognizable inputs, adversarial patches, universal perturbations, malware detection with discrete features